Network Intrusion Detection Using Machine Learning (Random Forest, Decision Tree and Logistic Regression on the UNSW-NB15 Dataset)

Problem Statement:

Computer networks are constantly targeted by attacks such as denial of service, exploits, fuzzing, reconnaissance and backdoors. Traditional rule-based (signature-based) intrusion detection systems compare network traffic with a fixed list of known attack patterns. They can therefore struggle to identify previously unseen or slowly evolving attacks, and keeping the rule set up to date requires continuous manual effort by security experts.

There is, therefore, a need for an automated and adaptive system that can analyse the statistical features of network traffic and classify each flow as Normal or Attack, so that suspicious activity can be flagged early for review by a security analyst. This project designs and implements a machine-learning-based Network Intrusion Detection System (NIDS) that performs data cleaning, feature analysis, preprocessing, model training and prediction on the labelled UNSW-NB15 dataset, compares three classifiers (Logistic Regression, Decision Tree and Random Forest), and evaluates the final model using standard classification metrics (Accuracy, Precision, Recall, F1-Score, Confusion Matrix). The project is an educational prototype and is not a replacement for a production intrusion detection system.

Objectives:

1. To obtain and load a suitable labelled network intrusion dataset (UNSW-NB15).
2. To understand the dataset's features, labels and class distribution.
3. To clean the dataset by handling missing values, infinite values, duplicate records and categorical features.
4. To preprocess the data using appropriate encoding (one-hot) and feature scaling.
5. To split the dataset into training and testing sets without data leakage.
6. To train and compare machine-learning classifiers (Logistic Regression, Decision Tree, Random Forest).
7. To classify network traffic as NORMAL (label 0) or ATTACK (label 1).
8. To evaluate the model using Accuracy, Precision, Recall, F1-score and the Confusion Matrix.
9. To visualise the class distribution, feature importance and model performance.
10. To save the trained model and preprocessing pipeline with Joblib for future predictions.
11. To demonstrate predictions on unseen test samples.

Software / Hardware Required:

• Operating System: macOS (MacBook Air, Apple M2)

• Environment / IDE: Jupyter Notebook (JupyterLab), Visual Studio Code

• Language: Python 3.x (virtual environment)

• Machine Learning: scikit-learn (Pipeline, ColumnTransformer, LogisticRegression, DecisionTreeClassifier, RandomForestClassifier)

• Data Handling: pandas, NumPy

• Visualisation: Matplotlib, Seaborn

• Model Persistence: Joblib

• Dataset: UNSW-NB15 (UNSW Canberra Cyber) — official training-set and testing-set CSV files, https://research.unsw.edu.au/projects/unsw-nb15-dataset

• Hardware: MacBook Air M2, 8 GB RAM or more, about 1 GB free disk space

Project Directory Structure:

The project ("NetworkIntrusionDetection") is organized with separate folders for the dataset, the Jupyter notebook, the reusable Python source files, the saved model, the generated outputs and the screenshots, as shown below.

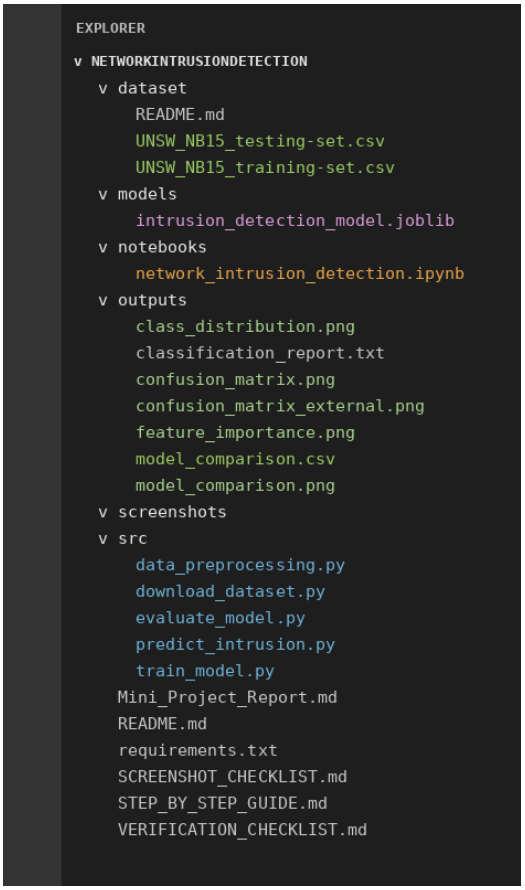


Fig 1: Step 1 — Project directory structure of NetworkIntrusionDetection (Explorer view)

Procedure:

1. Create the project folder “NetworkIntrusionDetection” and configure a Python virtual environment on macOS (python3 -m venv venv; source venv/bin/activate) (Step 1).

2. Install the required libraries (pandas, NumPy, scikit-learn, Matplotlib, Seaborn, Joblib, JupyterLab) using requirements.txt and verify the installation in the notebook (Step 2).

3. Download the UNSW-NB15 training-set and testing-set CSV files into the dataset folder using src/download_dataset.py (or manually from the official page) (Step 3).

4. Open notebooks/network_intrusion_detection.ipynb in Jupyter, load the training CSV and inspect its shape (175,341 × 45), column names, data types and sample records (Step 4).

5. Perform exploratory data analysis — missing and infinite values, duplicate records, class distribution, feature box plots — and verify that label 0 corresponds only to “Normal” and label 1 only to the nine attack categories of the attack_cat column (Step 5).

6. Clean the data: drop the id column and the attack_cat column (it reveals the answer, i.e. data leakage), convert infinite values to missing, and remove exact duplicate records; build the preprocessing pipeline — median imputation, log1p transform and standard scaling for numeric features, one-hot encoding for proto, service and state (Step 6).

7. Define the target label (0 = NORMAL, 1 = ATTACK) and split the data into 80 % training and 20 % testing sets using stratified sampling with a fixed random seed (Step 7).

8. Train Logistic Regression, Decision Tree and Random Forest classifiers, each inside the same scikit-learn Pipeline, using 5-fold stratified cross-validation on the training part only (Step 8).

9. Compare the models using the measured cross-validation scores and select the final model with the highest mean F1-Score (Step 9).

10. Fit the final model on the training set and evaluate it on the untouched test set: Accuracy, Precision, Recall, F1-Score, ROC-AUC, classification report, confusion matrix and feature importance (Step 10).

11. Evaluate the trained model additionally on the official UNSW-NB15 testing file, after removing records that already occur in the training data (Step 11).

12. Demonstrate intrusion prediction on unseen test records, showing the predicted class (NORMAL / ATTACK) with the attack probability (Step 12).

13. Save the trained pipeline and the feature-column order with Joblib (models/intrusion_detection_model.joblib), reload it, and confirm that it gives identical predictions (Step 13).

14. Save all charts, the classification report and the model comparison table in the outputs folder and prepare the project report (Step 14).

Implementation Code:

Key implementation modules of the project, taken from the project's source files (src/ folder) and executed in the notebook to produce the outputs shown in the next section.

1. Data Loading and Label Validation (src/data_preprocessing.py)

```
def load_raw_csv(path):
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(
            f"\nDataset file not found:\n  {path}\n\n"
            "How to fix it:\n"
            "  Option A (automatic) : python src/download_dataset.py\n"
            "  Option B (manual)    : follow dataset/README.md and place the two CSV\n"
            "                         files inside the 'dataset' folder of this project.\n"
        )
    # 'utf-8-sig' silently removes the invisible BOM character that this file
    # starts with. Without it the first column would be called '\ufeffid'.
    return pd.read_csv(path, encoding="utf-8-sig")


def validate_labels(df):
    for col in (TARGET_COL, ATTACK_CAT_COL):
        if col not in df.columns:
            raise KeyError(
                f"Column '{col}' not found. Columns present: {list(df.columns)[:10]} ...\n"
                "Check that you loaded the UNSW_NB15_training-set / testing-set CSV files "
                "(NOT the raw UNSW-NB15_1.csv ... files, which have no header)."
            )
    table = pd.crosstab(df[ATTACK_CAT_COL], df[TARGET_COL])
    normal_ok = table.loc["Normal"].get(1, 0) == 0
    attacks_ok = table.drop(index="Normal").get(0, pd.Series([0])).eq(0).all()
    if not (normal_ok and attacks_ok):
        raise ValueError("Label mapping is inconsistent:\n" + table.to_string())
    return table
```

2. Data Cleaning and Train-Test Split (src/data_preprocessing.py)

```
def clean_dataset(df, verbose=True):
    n_start = len(df)
    df = df.drop(columns=[c for c in (ID_COL, ATTACK_CAT_COL) if c in df.columns])
    df = df.replace([np.inf, -np.inf], np.nan)


    n_dups = int(df.duplicated().sum())
    df = df.drop_duplicates().reset_index(drop=True)


    if verbose:
        print(f"Rows before cleaning  : {n_start:,}")
        print(f"Duplicate rows removed: {n_dups:,}")
        print(f"Rows after cleaning   : {len(df):,}")


    y = df[TARGET_COL].astype(int)
    X = df.drop(columns=[TARGET_COL])
    return X, y


def split_data(X, y):
    return train_test_split(
        X, y, test_size=TEST_SIZE, stratify=y, random_state=RANDOM_STATE
    )
```

3. Preprocessing Pipeline and Classifiers (src/data_preprocessing.py)

```
def build_preprocessor(X):
    num_cols = numeric_columns(X)

    numeric_pipe = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("log1p", FunctionTransformer(np.log1p, feature_names_out="one-to-one")),
        ("scale", StandardScaler()),
    ])
    categorical_pipe = Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="missing")),
        ("onehot", OneHotEncoder(handle_unknown="infrequent_if_exist",
                                 min_frequency=50, sparse_output=False)),
    ])

    return ColumnTransformer(
        transformers=[
            ("num", numeric_pipe, num_cols),
            ("cat", categorical_pipe, CATEGORICAL_COLS),
        ],
        remainder="drop",
    )

def get_models():
    return {
        "Logistic Regression": LogisticRegression(
            max_iter=2000, class_weight="balanced", random_state=RANDOM_STATE),
        "Decision Tree": DecisionTreeClassifier(
            max_depth=15, class_weight="balanced", random_state=RANDOM_STATE),
        "Random Forest": RandomForestClassifier(
            n_estimators=200, class_weight="balanced_subsample",
            n_jobs=-1, random_state=RANDOM_STATE),
    }

def make_pipeline(X, classifier):
    return Pipeline([("preprocess", build_preprocessor(X)), ("model", classifier)])
```

4. Model Comparison by Cross-Validation (src/train_model.py)

```
def compare_models(X_train, y_train):
    cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)
    rows = []
    for name, clf in get_models().items():
        t0 = time.time()
        pipe = make_pipeline(X_train, clf)
        res = cross_validate(pipe, X_train, y_train, cv=cv, scoring=SCORING, n_jobs=1)
        rows.append({"model": name,
                     **{k: res[f"test_{k}"].mean() for k in SCORING},
                     "f1_std": res["test_f1"].std(),
                     "cv_time_sec": time.time() - t0})
        print(f"  {name:<20} mean F1 = {rows[-1]['f1']:.4f}   ({rows[-1]
['cv_time_sec']:.0f}s)")
    table = pd.DataFrame(rows)
    table.index = table["model"]
    table.index.name = None
    return table


# Model selection and final training (inside main())
best_name = cv_table.sort_values(["f1", "recall"], ascending=False).index[0]
final_pipe = make_pipeline(X_train, get_models()[best_name])
final_pipe.fit(X_train, y_train)
```

5. Evaluation Metrics and Confusion Matrix (src/evaluate_model.py)

```
def compute_metrics(y_true, y_pred, y_score=None):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    m = {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, pos_label=1, zero_division=0),
        "recall": recall_score(y_true, y_pred, pos_label=1, zero_division=0),
        "f1_score": f1_score(y_true, y_pred, pos_label=1, zero_division=0),
        "false_positive_rate": fp / (fp + tn) if (fp + tn) else float("nan"),
        "TN": int(tn), "FP": int(fp), "FN": int(fn), "TP": int(tp),
        "n_samples": int(len(y_true)),
    }
    if y_score is not None:
        m["roc_auc"] = roc_auc_score(y_true, y_score)
    return {k: (float(v) if isinstance(v, (float, np.floating)) else v) for k, v in m.items()}


def evaluate_pipeline(pipeline, X, y):
    y_pred = pipeline.predict(X)
    y_score = pipeline.predict_proba(X)[:, 1] if hasattr(pipeline, "predict_proba") else None
    return compute_metrics(y, y_pred, y_score), y_pred


def plot_confusion_matrix(y_true, y_pred, title, path=None):
    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
    fig, ax = plt.subplots(figsize=(5.5, 4.5))
    sns.heatmap(cm, annot=True, fmt=",d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=TARGET_NAMES, yticklabels=TARGET_NAMES, annot_kws={"size": 14})
    ax.set_xlabel("Predicted label")
    ax.set_ylabel("Actual label")
    ax.set_title(title)
    fig.tight_layout()
    if path:
        fig.savefig(path, dpi=150)
    return fig
```

6. Saving, Loading and Prediction (src/train_model.py, src/predict_intrusion.py)

```
# Saving the trained pipeline (inside main() of train_model.py)
bundle = {"pipeline": final_pipe, "model_name": best_name,
          "feature_columns": list(X.columns), "class_names": CLASS_NAMES,
          "sklearn_version": sklearn.__version__}
joblib.dump(bundle, MODEL_PATH, compress=3)


# Loading and predicting (predict_intrusion.py)
def predict_dataframe(bundle, df):
    cols = bundle["feature_columns"]
    missing = [c for c in cols if c not in df.columns]
    if missing:
        raise KeyError(
            f"The input data is missing {len(missing)} required column(s): {missing[:8]} ...\
n"
            f"The model needs exactly these {len(cols)} columns: {cols}")
    X = df[cols]                                    # re-order columns exactly like training
    pipe = bundle["pipeline"]
    pred = pipe.predict(X)
    prob_attack = pipe.predict_proba(X)[:, 1]
    out = pd.DataFrame({
        "prediction": [CLASS_NAMES[int(p)] for p in pred],
        "attack_probability": prob_attack.round(4),
    }, index=df.index)
    return out
```

Output (Step-by-Step Screenshots):

```
Python      : 3.12.3
Platform    : Linux-6.18.44-fc-v114-x86_64-with-glibc2.39
pandas      : 3.0.2
numpy       : 2.4.4
scikit-learn: 1.8.0
matplotlib  : 3.10.8
seaborn     : 0.13.2
joblib      : 1.5.3
Project root: /home/claude/build/NetworkIntrusionDetection
```

Fig 2: Step 2 — Python environment and library verification (Python, pandas, NumPy, scikit-learn, Matplotlib, Seaborn, Joblib)

```
FOUND   /home/claude/build/NetworkIntrusionDetection/dataset/UNSW_NB15_training-set.csv
FOUND   /home/claude/build/NetworkIntrusionDetection/dataset/UNSW_NB15_testing-set.csv

Dataset loaded successfully!
```

Fig 3: Step 3 — Dataset files located and the UNSW-NB15 training set loaded successfully in Jupyter Notebook

```
Shape (rows, columns): (175341, 45)

Column names and data types:
id                    int64       dwin                  int64
dur                 float64       tcprtt              float64
proto                   str       synack              float64
service                 str       ackdat              float64
state                   str       smean                 int64
spkts                 int64       dmean                 int64
dpkts                 int64       trans_depth           int64
sbytes                int64       response_body_len     int64
dbytes                int64       ct_srv_src            int64
rate                float64       ct_state_ttl          int64
sttl                  int64       ct_dst_ltm            int64
dttl                  int64       ct_src_dport_ltm      int64
sload               float64       ct_dst_sport_ltm      int64
dload               float64       ct_dst_src_ltm        int64
sloss                 int64       is_ftp_login          int64
dloss                 int64       ct_ftp_cmd            int64
sinpkt              float64       ct_flw_http_mthd      int64
dinpkt              float64       ct_src_ltm            int64
sjit                float64       ct_srv_dst            int64
djit                float64       is_sm_ips_ports       int64
swin                  int64       attack_cat              str
stcpb                 int64       label                 int64
dtcpb                 int64
```

Fig 4: Step 4 — Dataset shape (175,341 rows × 45 columns), column names and data types

In [4]:
```
df.head()
```

```
   id       dur proto service state  spkts  dpkts  sbytes  dbytes       rate  \
0   1  0.121478   tcp       -   FIN      6      4     258     172  74.087490
1   2  0.649902   tcp       -   FIN     14     38     734   42014  78.473372
2   3  1.623129   tcp       -   FIN      8     16     364   13186  14.170161
3   4  1.681642   tcp     ftp   FIN     12     12     628     770  13.677108
4   5  0.449454   tcp       -   FIN     10      6     534     268  33.373826

   sttl  dttl         sload          dload  sloss  dloss      sinpkt  \
0   252   254  14158.942380    8495.365234      0      0   24.295600
1    62   252   8395.112305  503571.312500      2     17   49.915000
2    62   252   1572.271851   60929.230470      1      6  231.875571
3    62   252   2740.178955    3358.622070      1      3  152.876547
4   254   252   8561.499023    3987.059814      2      1   47.750333

       dinpkt          sjit          djit  swin       stcpb       dtcpb  dwin  \
0    8.375000     30.177547     11.830604   255   621772692  2202533631   255
1   15.432865     61.426934   1387.778330   255  1417884146  3077387971   255
2  102.737203  17179.586860  11420.926230   255  2116150707  2963114973   255
3   90.235726    259.080172   4991.784669   255  1107119177  1047442890   255
4   75.659602   2415.837634    115.807000   255  2436137549  1977154190   255

     tcprtt    synack    ackdat  smean  dmean  trans_depth  response_body_len  \
0  0.000000  0.000000  0.000000     43     43            0                  0
1  0.000000  0.000000  0.000000     52   1106            0                  0
2  0.111897  0.061458  0.050439     46    824            0                  0
3  0.000000  0.000000  0.000000     52     64            0                  0
4  0.128381  0.071147  0.057234     53     45            0                  0

   ct_srv_src  ct_state_ttl  ct_dst_ltm  ct_src_dport_ltm  ct_dst_sport_ltm  \
0           1             0           1                 1                 1
1          43             1           1                 1                 1
2           7             1           2                 1                 1
3           1             1           2                 1                 1
4          43             1           2                 2                 1

   ct_dst_src_ltm  is_ftp_login  ct_ftp_cmd  ct_flw_http_mthd  ct_src_ltm  \
0               1             0           0                 0           1
1               2             0           0                 0           1
2               3             0           0                 0           2
3               3             1           1                 0           2
4              40             0           0                 0           2

   ct_srv_dst  is_sm_ips_ports attack_cat  label
0           1               0     Normal      0
1           6               0     Normal      0
2           6               0     Normal      0
3           1               0     Normal      0
4          39               0     Normal      0
```

Fig 5: Step 4 — First five records of the dataset (df.head())

```
Missing values in the whole file : 0
Infinite values                  : 0
Duplicate rows (ignoring id & attack_cat): 74072

Unique values in categorical columns:
proto      133
service     13
state        9
```

In [6]:

```
check = validate_labels(df)      # raises an error if the mapping is inconsistent
print("Label mapping verified: label 0 == 'Normal' only, label 1 == attack families only\n")
check
```

```
Label mapping verified: label 0 == 'Normal' only, label 1 == attack families only

label               0      1
attack_cat
Analysis            0   2000
Backdoor            0   1746
DoS                 0  12264
Exploits            0  33393
Fuzzers             0  18184
Generic             0  40000
Normal          56000      0
Reconnaissance      0  10491
Shellcode           0   1133
Worms               0    130
```

Fig 6: Step 5 — Exploratory data analysis: missing, infinite and duplicate counts, categorical columns, and verification of the label mapping (attack_cat vs label)

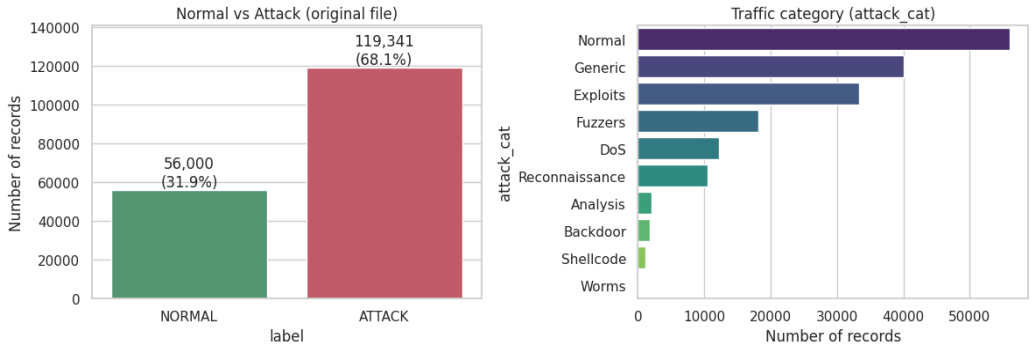
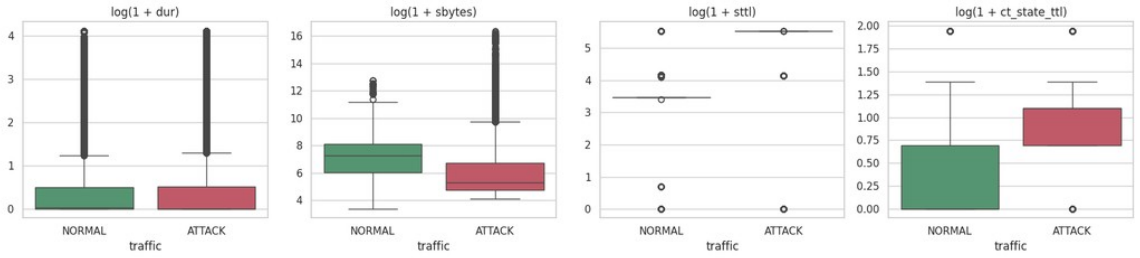


Fig 7: Step 5 — Class distribution: Normal vs Attack records and traffic categories in the original training file




```
Attack share (%) among the 6 most common 'proto' values:
proto
unas    100.0
sctp    100.0
ospf     97.5
udp      78.0
tcp      51.1
arp       0.0

Attack share (%) among the 6 most common 'state' values:
state
INT    93.1
REQ    53.5
FIN    52.2
RST    14.5
CON     8.0
ECO     0.0
```

Fig 8: Step 5 — Feature comparison between Normal and Attack traffic (box plots) and attack share per protocol and state

```
Rows before cleaning  : 175,341
Duplicate rows removed: 74,072
Rows after cleaning   : 101,269

Target mapping: {0: 'NORMAL', 1: 'ATTACK'}
Class counts after cleaning:
label
NORMAL    51890
ATTACK    49379

Number of feature columns: 42
Categorical features: ['proto', 'service', 'state']
Numeric features    : 39

Training set: 81,015 rows | Test set: 20,254 rows

Attack share in training set: 48.76 %
Attack share in test set    : 48.76 %
```

Fig 9: Step 6/7 — Data cleaning (duplicates removed), target-label mapping and stratified 80/20 train-test split

```
Before preprocessing: (81015, 42) -> after: (81015, 62)
(42 original features became 62 columns after one-hot encoding)
```

Fig 10: Step 6 — Preprocessing pipeline: 42 features transformed into 62 columns (median imputation, log1p, scaling, one-hot encoding), fitted on training data only

```
Logistic Regression  finished in    13 s
Decision Tree        finished in     9 s
Random Forest        finished in   124 s

                     accuracy  precision  recall      f1  f1_std  cv_time_sec
Logistic Regression    0.9034     0.8638  0.9519  0.9057  0.0030      13.2499
Decision Tree          0.9174     0.8999  0.9346  0.9169  0.0020       9.1032
Random Forest          0.9263     0.9083  0.9442  0.9259  0.0016     123.6584
```

Fig 11: Step 8 — Training Logistic Regression, Decision Tree and Random Forest with 5-fold cross-validation

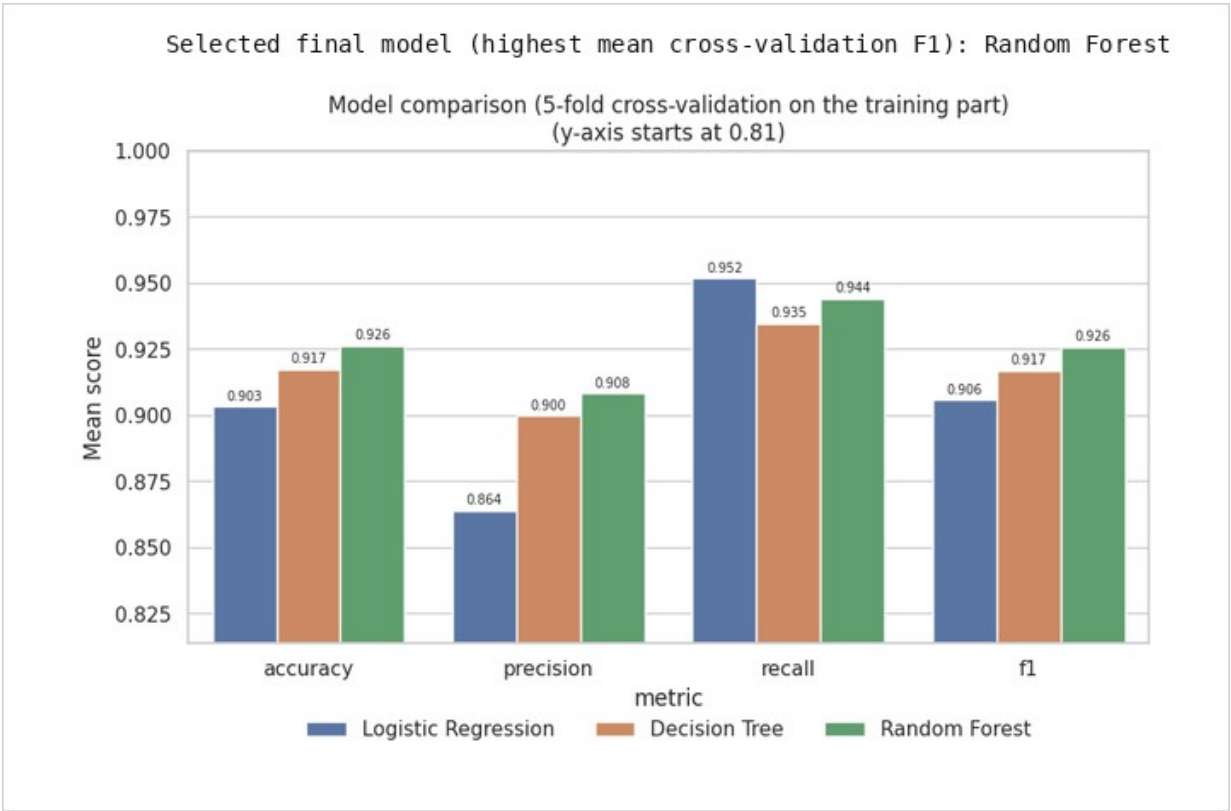


Fig 12: Step 9 — Model comparison chart and selection of the final model (Random Forest)

```
=== Random Forest - hold-out test set ===
Samples   : 20,254
Accuracy  : 0.9301  (93.01 %)
Precision : 0.9135  (ATTACK class)
Recall    : 0.9463  (ATTACK class)
F1-score  : 0.9296  (ATTACK class)
ROC-AUC   : 0.9857
False-positive rate: 0.0853
```

	precision	recall	f1-score	support
NORMAL	0.9471	0.9147	0.9306	10378
ATTACK	0.9135	0.9463	0.9296	9876
accuracy			0.9301	20254
macro avg	0.9303	0.9305	0.9301	20254
weighted avg	0.9307	0.9301	0.9301	20254

Fig 13: Step 10 — Final model evaluation on the hold-out test set: Accuracy, Precision, Recall, F1-Score, ROC-AUC and classification report

```
Attacks correctly detected (TP): 9,346   | Attacks MISSED (FN): 530
Normal traffic correctly passed (TN): 9,493 | False alarms (FP): 885
```

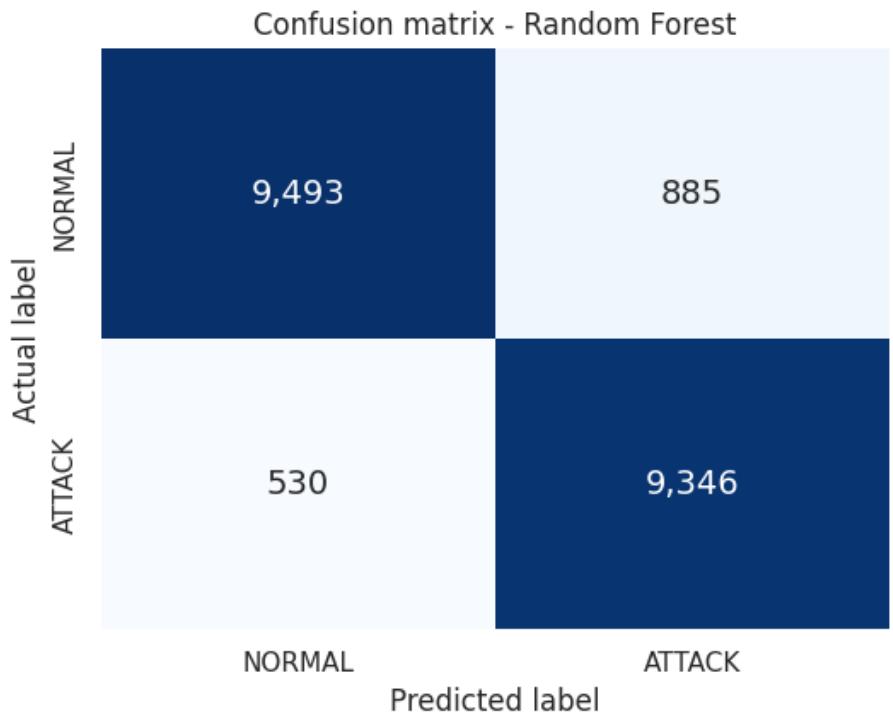


Fig 14: Step 10 — Confusion matrix of the Random Forest model on the hold-out test set

```
sttl              0.1297
ct_state_ttl      0.0975
dload             0.0629
dttl              0.0614
ackdat            0.0475
synack            0.0459
tcprtt            0.0391
dinpkt            0.0372
dmean             0.0355
ct_srv_dst        0.0336
sload             0.0316
rate              0.0305
ct_dst_src_ltm    0.0273
sbytes            0.0260
ct_srv_src        0.0258
```

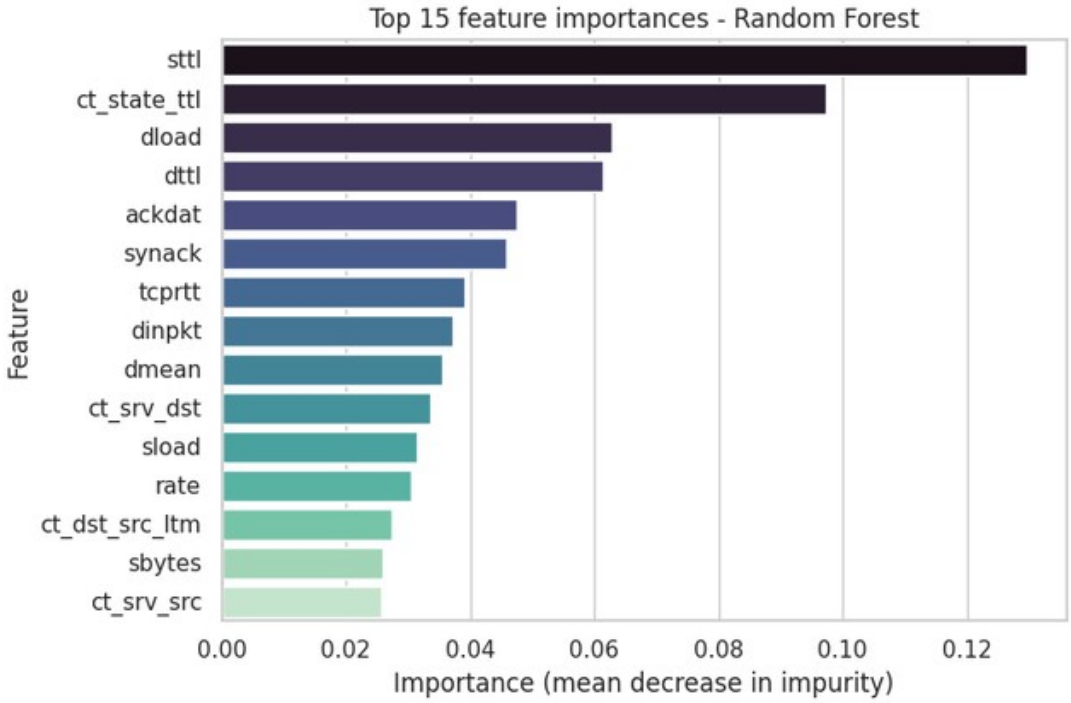


Fig 15: Step 10 — Top 15 feature importances of the Random Forest model

```
Official testing file after cleaning: 53,952 rows; 1,304 also occur in training data and were
    removed -> 52,648 rows used
=== Random Forest - official testing file (external) ===
Samples   : 52,648
Accuracy  : 0.8244  (82.44 %)
Precision : 0.6795  (ATTACK class)
Recall    : 0.9711  (ATTACK class)
F1-score  : 0.7995  (ATTACK class)
ROC-AUC   : 0.9628
False-positive rate: 0.2583
```

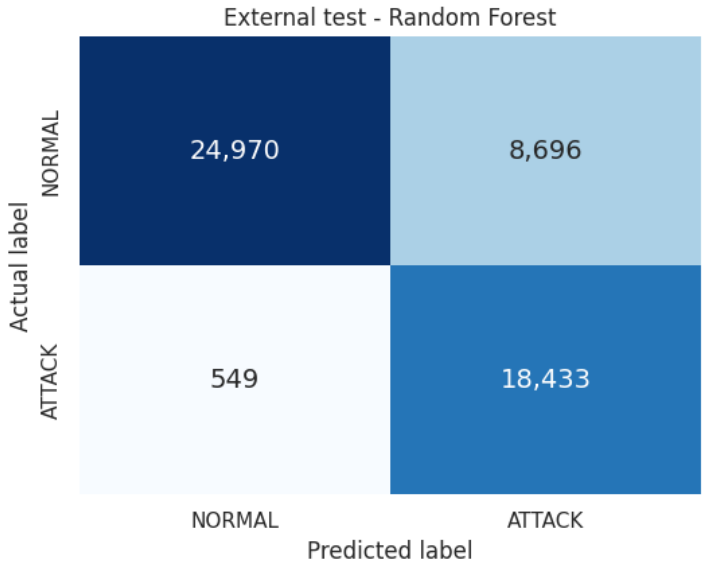


Fig 16: Step 11 — External evaluation on the official UNSW-NB15 testing file (records already present in training removed)

	actual	predicted	attack_probability	proto	service	state	sttl	\
44799	NORMAL	ATTACK	0.675	tcp	-	FIN	254	
28884	NORMAL	NORMAL	0.000	tcp	-	FIN	31	
9596	NORMAL	NORMAL	0.000	tcp	-	FIN	31	
4507	NORMAL	NORMAL	0.000	tcp	-	FIN	31	
40644	NORMAL	NORMAL	0.370	tcp	-	FIN	254	
63099	ATTACK	ATTACK	0.940	tcp	http	FIN	62	
46736	ATTACK	ATTACK	0.650	tcp	-	FIN	254	
65092	ATTACK	ATTACK	1.000	udp	-	INT	254	
63050	ATTACK	ATTACK	0.850	tcp	http	FIN	62	
53710	ATTACK	ATTACK	0.990	udp	-	INT	254	

	ct_state_ttl	correct
44799	1	False
28884	0	True
9596	0	True
4507	0	True
40644	1	True
63099	1	True
46736	1	True
65092	2	True
63050	1	True
53710	2	True

Fig 17: Step 12 — Predictions on ten randomly chosen unseen test records (5 NORMAL, 5 ATTACK)

```
Actual label        : NORMAL
Predicted label     : NORMAL
P(attack) / P(normal): 0.000 / 1.000
proto           tcp
service           -
state           FIN
dur        0.016861
spkts            40
dpkts            42
sbytes         2542
dbytes        23508
sttl             31
dttl             29
```

Fig 18: Step 12 — Prediction result for a NORMAL network traffic sample

```
Actual label        : ATTACK
Predicted label     : ATTACK
P(attack) / P(normal): 0.940 / 0.060
proto           tcp
service        http
state           FIN
dur        0.725329
spkts            10
dpkts            10
sbytes          788
dbytes         1274
sttl             62
dttl            252
```

Fig 19: Step 12 — Prediction result for an ATTACK network traffic sample

```
Saved: /home/claude/build/NetworkIntrusionDetection/models/intrusion_detection_model.joblib
    (39.7 MB)

Loaded model: Random Forest | trained with scikit-learn 1.8.0
Predictions identical to the in-memory model on all 20254 test rows: True
Hold-out accuracy of the reloaded model: 0.9301
```

Fig 20: Step 13 — Trained pipeline saved with Joblib and reloaded: predictions identical on all 20,254 test records

```
Files in outputs/:
  class_distribution.png              84.5 KB
  classification_report.txt            1.6 KB
  confusion_matrix.png                35.1 KB
  confusion_matrix_external.png       34.8 KB
  feature_importance.png              57.6 KB
  model_comparison.csv                 0.2 KB
  model_comparison.png                71.5 KB

Model file: intrusion_detection_model.joblib  39.7 MB
```

Fig 21: Step 14 — Generated output files (charts, classification report, comparison table) and the saved model

Result:

A machine-learning-based Network Intrusion Detection System (NIDS) was successfully designed and implemented in a Jupyter Notebook using scikit-learn pipelines on the UNSW-NB15 dataset. After removing 74,072 duplicate records, 101,269 records (51,890 Normal and 49,379 Attack) with 42 features were split in a stratified 80:20 ratio. Logistic Regression, Decision Tree and

Random Forest were compared using 5-fold cross-validation on the training part, and the Random Forest (mean cross-validation F1-Score 0.9259) was selected as the final model. On the held-out test set of 20,254 records it achieved an Accuracy of 93.01%, Precision of 91.35%, Recall of 94.63%, F1-Score of 92.96% and ROC-AUC of 0.9857; it detected 9,346 of 9,876 attacks (530 missed) and raised 885 false alarms on 10,378 normal records. The trained pipeline (preprocessing and classifier) was saved with Joblib and reloaded to classify unseen network-traffic samples as NORMAL or ATTACK with an attack probability.

As an additional check on the separate official UNSW-NB15 testing file (52,648 records after removing duplicates and records already present in training), the model obtained an Accuracy of 82.44%, Precision of 67.95%, Recall of 97.11% and F1-Score of 79.95%. Recall stays high, but the false-positive rate rises to 25.83% because this partition has a different traffic distribution, which shows that a learned detector must be re-evaluated on traffic that differs from its training data. This project is an educational prototype and not a replacement for a production intrusion detection system.